



UNIVERSITÉ HASSAN 1er
FACULTÉ DE SCIENCES ET TECHNIQUES
— SETTAT —



Projet de Fin d'Etude

En vue de l'obtention du diplôme

MASTER SCIENCES ET TECHNIQUES

Spécialité : Réseaux et Systèmes Informatiques (RSI)

Présenté par :
MOGHANDEZ Abdellah

Conception et Réalisation d'un Agent IA de Diagnostic Technique pour le Système d'Information CCU via LangGraph et Neo4j GraphRAG

inetum.

Sous la direction de :

Mr. Mohamed Amine Brakez, Chef de projet

Soutenu le 25 juin 2026, devant le jury :

Année Universitaire : 2025 - 2026

Remerciements

Avant tout, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce stage et à la réalisation de ce rapport.

À l'entreprise Inetum Maroc : Je remercie chaleureusement l'ensemble des équipes de Inetum pour leur accueil, leur confiance et leur disponibilité tout au long de mon stage. Un remerciement particulier à mon maître de stage, **Mohamed Amine BRAKEZ**, pour ses conseils avisés et son soutien constant, qui ont considérablement enrichi mon expérience professionnelle et académique.

À l'équipe pédagogique : Je suis reconnaissante envers mes enseignants et encadrants universitaires pour la qualité de leur formation, leur accompagnement et leurs orientations, qui ont constitué un guide essentiel dans la réalisation de ce projet.

À mes proches : Je dédie une pensée particulière à ma famille et à mes amis pour leur soutien indéfectible, leur patience et leurs encouragements tout au long de ce parcours exigeant.

À mes camarades : je remercie mes collègues de promotion pour les échanges enrichissants, l'entraide et la bonne humeur partagée, qui ont rendu cette expérience encore plus stimulante.

Résumé

Ce rapport présente les travaux réalisés dans le cadre d'un stage assistant ingénieur effectué au sein d'**Inetum Offshore Maroc**. Le projet porte sur la conception et la réalisation d'un **Système Multi Agents de diagnostic technique** destiné au système d'information **CCU** (Commande et Catalogue Unifiés) d'un opérateur télécom.

L'objectif est d'accélérer et de fiabiliser le traitement des incidents techniques en corrélant automatiquement plusieurs sources de données : les **logs réseau**, le **contexte client** issu du CRM et les **tickets historiques** stockés dans un graphe de connaissances **Neo4j**. Le système repose sur une orchestration agentique avec **LangGraph**, une interface d'embedding extensible et un modèle de langage local via **Ollama**.

La première partie du travail a concerné la conception de l'architecture agentique : parsing de l'incident, collecte parallèle des sources, raisonnement sur la cause racine, proposition de remédiation, guardrail de contenu, génération de rapport PDF et notification. La seconde partie a porté sur la réalisation technique : implémentation Python avec **FastAPI**, base de graphe **Neo4j** incluant un index vectoriel, ingestion des données mockées, tests avec **pytest** et conteneurisation via **Docker Compose**.

Ce projet m'a permis d'appréhender les enjeux de l'IA générative appliquée au support télécom, la conception de systèmes multi-agents, la recherche augmentée par graphe (**GraphRAG**) et l'implémentation d'une application LLM dans un environnement local et contrôlé, tout en respectant une contrainte forte : aucune action corrective n'est exécutée automatiquement sur un système réel.

Mots clés : Agent IA, Diagnostic technique, CCU, LangGraph, Neo4j, GraphRAG, FastAPI, TMF620, TMF622, LLM, Ollama.

Abstract

This report presents the work carried out as part of an assistant engineer internship at **Inetum Offshore Maroc**. The project focuses on the design and implementation of a **multi-agent system for technical diagnosis** targeting the **CCU** (Commande et Catalogue Unifiés) information system of a telecom operator.

The objective is to accelerate and improve the reliability of technical incident handling by automatically correlating multiple data sources: **network logs**, **customer context** from the CRM, and **historical tickets** stored in a **Neo4j** knowledge graph. The system relies on an agentic orchestration built with **LangGraph**, an extensible embedding interface, and a local language model via **Ollama**.

The first part of the work focused on designing the agentic architecture: incident parsing, parallel source collection, root-cause reasoning, remediation proposal, content guardrail, PDF report generation, and notification. The second part covered the technical implementation: Python implementation with **FastAPI**, **Neo4j** graph database including a vector index, ingestion of mocked data, **pytest** testing, and containerization via **Docker Compose**.

This project allowed me to address the challenges of generative AI applied to telecom support, multi-agent system design, graph-augmented retrieval (**GraphRAG**), and the implementation of an LLM-based application in a controlled local environment, while respecting a strong constraint: no corrective action is ever executed automatically on a real system.

Keywords: Multi-Agent System, Technical Diagnosis, CCU, LangGraph, Neo4j, GraphRAG, FastAPI, TMF620, TMF622, LLM, Ollama, Telecom Support.

Table des matières

Table des figures

Liste des tableaux

Liste des sigles et acronymes

API	Application Programming Interface
BSS	Business Support Systems
CCU	Commande et Catalogue Unifiés
CRM	Customer Relationship Management
ESN	Entreprise de Services du Numérique
FastAPI	Framework web Python pour APIs REST
FTTH	Fiber To The Home
GBL	Global Business Line
GraphRAG	Graph Retrieval-Augmented Generation
IOM	Inetum Offshore Maroc
LLM	Large Language Model
MOCK	Mode de simulation/déterministe
Neo4j	Système de gestion de base de données graphe
OSP	Outside Plant
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
SI	Système d'Information
SID	Shared Information/Data (TM Forum)
SSE	Server-Sent Events
TMF	TM Forum
UI	User Interface

Introduction Générale

Dans un secteur des télécommunications en mutation permanente, la qualité du support client et la rapidité de résolution des incidents techniques constituent des leviers stratégiques de différenciation. Les opérateurs modernes doivent gérer des systèmes d'information de plus en plus complexes, où les données pertinentes à un incident sont dispersées entre logs réseau, bases clients, historiques de tickets et catalogues de produits. Face à cette fragmentation, le diagnostic repose encore trop souvent sur une expertise humaine et une investigation manuelle, sources de lenteur et d'erreur.

Ce mémoire présente le travail réalisé au sein de **Inetum Offshore Maroc**, dans le cadre du projet de fin d'études du Master Réseaux et Systèmes Informatiques. L'objectif était de concevoir et de développer un **agent IA de diagnostic technique** pour le système d'information **CCU** (Commande et Catalogue Unifiés) d'un opérateur télécom, en s'appuyant sur les APIs **TMF620** (Product Catalog) et **TMF622** (Product Ordering).

L'agent proposé corrèle automatiquement trois familles de sources : les **logs réseau**, le **contexte client** (CRM) et les **tickets historiques** indexés dans un graphe de connaissances **Neo4j**. Il produit un diagnostic structuré, une explication de la cause racine, une fiche de remédiation informative, un rapport PDF et une notification, tout en garantissant qu'**aucune action corrective n'est jamais exécutée automatiquement** sur un système réel. Un garde-rail de contenu assure le filtrage des informations personnelles avant tout envoi externe.

La première partie de ce rapport situe le contexte du projet, la problématique métier et les objectifs poursuivis. La deuxième partie détaille la conception de l'architecture agentique, le modèle de données du graphe et le mécanisme de recherche augmentée par graphe (GraphRAG). La troisième partie présente la réalisation technique : orchestration LangGraph, implémentation des agents, API FastAPI et génération des rapports. Enfin, la dernière partie traite de la stratégie de validation, du déploiement avec Docker et d'un scénario de démonstration de bout en bout.

Présentation du Projet

Introduction

Ce chapitre se focalise sur l'introduction de l'entreprise d'accueil ainsi que sur la contextualisation du projet. Il débutera par une présentation de l'entreprise hôte et de ses domaines d'activité. Ensuite, nous aborderons la formulation de la problématique, les objectifs du projet, et enfin, nous discuterons de la méthodologie de gestion de projet et de sa planification.

1.1 Présentation de l'organisme d'accueil

Cette première partie présente le groupe Inetum, ses chiffres clés ainsi que sa structure organisationnelle à l'échelle internationale et nationale.

1.1.1 Le Groupe Inetum

Inetum est une ESN (Entreprise de Services du Numérique) agile, une société de services et de solutions digitales, et un groupe international qui aide les entreprises et les institutions à tirer le meilleur du digital flow. Aussi acteur européen de référence des services informatiques à valeur ajoutée et des logiciels.

À l'international, le Groupe compte 20 filiales en France, Belgique, Luxembourg, Suisse, Espagne, Portugal, Angleterre, Pologne, Roumanie, Singapour, Dubaï, Maroc, Côte d'Ivoire, Cameroun, Angola, Brésil, Colombie, Mexique, USA, Pérou et Panama. Avec son profil de multi-spécialiste, Inetum met au service de ses clients une combinaison unique de proximité, d'organisation sectorielle et de solutions de qualité industrielle. Le groupe qui compte plus de 28.000 collaborateurs a réalisé en 2024 un chiffre d'affaires de 2,5 M€. Elle est implantée dans toute la France autour de trois grandes régions et avec plus de 40 agences.

Le tableau ci-dessous regroupe les informations d'identité et les chiffres clés du groupe au niveau mondial et local :

Fiche d'identité globale	
Date de création	1970
Direction	Vincent ROUAIX, Président-directeur Général
Siège social	145, Boulevard Victor Hugo, 93 400 Saint-Ouen, FRANCE
Actionnariat	Bain Capital Private Equity (Majoritaire depuis 2022)
Chiffres clés 2024	
Chiffre d'affaires	2,5 Md€ (Global) / 40 M€ (Maroc)
Effectifs	28 000 (Monde) / +800 (Maroc)
Présence	19 pays, 130 bureaux

TABLE 1.1 : Identité et chiffres clés d'Inetum

1.1.2 Inetum Offshore Maroc (IOM)

Filiale à 100% d'Inetum, Inetum Offshore Maroc est le vecteur de développement du Groupe dans les pays francophones du continent africain. Elle est le leader incontesté au Maroc en matière d'intégration et de mise en œuvre de progiciels de gestion.

IOM met à la disposition des entreprises, ses consultants métiers et son savoir-faire et propose à ses clients un service complet qui couvre toutes les phases du cycle de vie des projets :

- Conseil,
- Conception,
- Intégration,
- Formation,
- Déploiement et Maintenance.

Le tableau suivant représente la fiche technique d'Inetum Offshore Maroc, qui résume toutes ses informations :

Date de création	2011
Direction	OUALI Fouzia
Taille de l'entreprise	1000 collaborateurs
Siège social	Casa Nearshore Parc, Shore 28. 100, Bd. Al Qods, Sidi Maârouf, Casablanca
Forme juridique	Société Anonyme (SA)
Capitale	400 000 DHS

TABLE 1.2 : Fiche technique d'Inetum Offshore Maroc

1.1.3 Organisation et pôles d'expertise

Inetum structure son expertise autour de quatre "Global Business Lines" (GBL) majeures :

- **Inetum Consulting (2% du CA) :** Conseil en stratégie digitale et IA.
- **Inetum Technologies (64% du CA) :** Gestion des infrastructures, Cloud et Cybersécurité.
- **Inetum Solutions (26% du CA) :** Intégration d'ERP (SAP, Microsoft, Sage).
- **Inetum Software (8% du CA) :** Édition de logiciels verticaux.

1.1.4 Secteurs d'activité

Inetum est l'un des leaders mondiaux dans le domaine du conseil, des services informatiques et de l'infogérance. Les métiers du groupe s'organisent en 6 grandes disciplines

- **Conseil :** Accompagnement stratégique.
- **Services applicatifs et d'infrastructures :** Support et développement.
- **Intégration systèmes :** Business Solutions, ERP, CRM.
- **Outsourcing :** Infogérance et gestion externe.
- **Value Added Reselling :** Revente de solutions à valeur ajoutée.
- **Software :** Solutions verticales (Assurances, Santé) et transverses (Chronotime).

Le groupe intervient dans l'ensemble des secteurs, pour répondre aux besoins de ses clients et leur proposer les solutions les mieux adaptées à leurs métiers spécifiques. Le groupe dispose d'une stratégie pertinente et d'un savoir-faire très solide pour développer ses relations clients dans plusieurs secteurs.

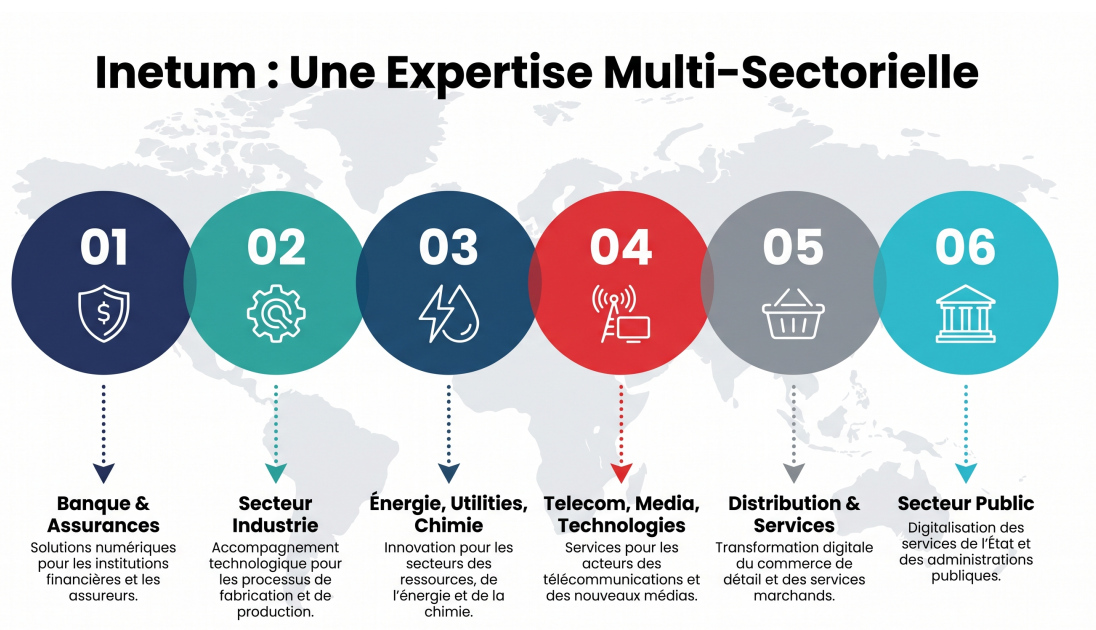


FIGURE 1.1 : Secteurs d'activité

1.1.5 Organigramme d'Inetum Offshore Maroc

Dans le cadre de notre projet, il est essentiel de bien comprendre la structure organisationnelle de l'équipe projet. L'organigramme joue un rôle crucial en définissant les rôles et responsabilités de chaque membre, en facilitant la communication et en assurant une coordination efficace des tâches. Cet organigramme permet également de clarifier les relations hiérarchiques et fonctionnelles au sein de l'équipe.

L'organigramme détaillé du projet est présenté dans la figure ?? :

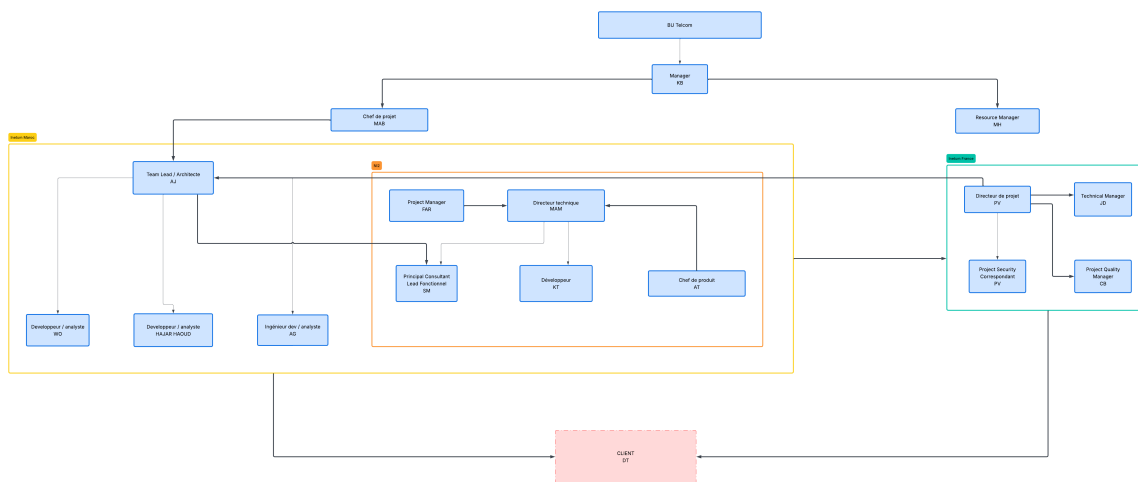


FIGURE 1.2 : Organigramme d'Inetum Offshore Maroc

1.1.6 Partenaires et Clients

Inetum collabore avec des partenaires technologiques de premier plan afin d'enrichir son offre de services et de répondre efficacement aux besoins de ses clients. Grâce à cette stratégie et à son expertise locale, plus de 500 entreprises marocaines lui font confiance pour les accompagner dans leur transformation digitale.

Partenaires stratégiques :

- **SAP** : Solutions ERP pour la gestion d'entreprise.
- **Microsoft** : Technologies cloud et bureautique.
- **Oracle** : Bases de données et logiciels métiers.
- **IBM** : Services IT, IA et infrastructure.

Principaux clients au Maroc :

- **OCP** : Leader mondial dans le secteur des phosphates.
- **ONCF** : Opérateur national du transport ferroviaire.
- **Allianz** : Compagnie d'assurance internationale.
- **Royal Air Maroc** : Compagnie aérienne nationale.

1.2 Contexte du projet

1.2.1 Cadre général du projet

Dans un secteur des télécommunications en constante mutation, les opérateurs doivent moderniser leur Système d'Information (SI) Commercial pour répondre aux attentes des clients et soutenir le déploiement de nouvelles technologies (FTTH, 4G/5G). Le programme de refonte porte sur le système **CCU** (Commande et Catalogue Unifiés), qui centralise le catalogue de produits et la gestion des commandes clients.

Le projet objet de ce mémoire ne porte pas sur la migration de données, mais sur la conception d'un **agent IA de diagnostic technique** capable d'assister les équipes de support dans la résolution des incidents affectant le SI CCU. L'agent exploite les APIs **TMF620** (Product Catalog) et **TMF622** (Product Ordering) du TM Forum pour s'aligner sur les standards de l'industrie télécom.

1.2.2 Problématique et enjeux

Les équipes de support technique font face à des incidents dont l'investigation requiert de consulter plusieurs sources disjointes :

- Les **logs réseau** et applicatifs, souvent volumineux et difficiles à interpréter rapidement.
- Le **contexte client** stocké dans le CRM (statut du contrat, services actifs, historique).
- Les **tickets historiques** résolus, qui contiennent une expertise précieuse mais sont difficiles à exploiter manuellement.

Cette dispersion des données engendre des temps de diagnostic longs, une dépendance forte à l'expertise humaine et un risque d'erreur. La problématique centrale est donc la suivante : **comment concevoir un système intelligent capable de corrélérer automatiquement ces sources pour produire un diagnostic fiable, explicable et actionnable, sans jamais exécuter d'action corrective automatique sur le SI réel ?**

1.2.3 Périmètre fonctionnel

Le périmètre du projet couvre les fonctionnalités suivantes :

1. **Intake intelligent** : normalisation d'un incident exprimé en langage naturel ou structuré.
2. **Collecte multi-source** : récupération parallèle des logs, du contexte client et des tickets similaires.
3. **Raisonnement sur la cause racine** : corrélation des sources avec attribution d'un niveau de confiance.
4. **Gestion des tickets** : mapping ou création d'un ticket dans l'outil de ticketing (Zammad).
5. **Remédiation explicative** : proposition d'actions informatives (non exécutables automatiquement).
6. **Guardrail de contenu** : vérification PII avant envoi externe.

7. **Génération de rapport** : production d'un rapport PDF/HTML.

8. **Notification** : envoi d'email et ajout d'une note dans le ticket.

1.2.4 Objectifs du projet

Les objectifs du projet se déclinent en quatre axes :

- **Centraliser l'investigation** : offrir un point d'entrée unique qui agrège les sources pertinentes à un incident.
- **Fiabiliser le diagnostic** : réduire les erreurs humaines grâce à une corrélation structurée et à un raisonnement guidé par les données.
- **Capitaliser l'expertise** : exploiter les tickets historiques via un graphe de connaissances et une recherche sémantique.
- **Garantir la sécurité** : s'assurer qu'aucune action technique n'est exécutée automatiquement et que les données sensibles sont filtrées.

1.3 Méthodologie et planification

1.3.1 Choix méthodologique

La conduite du projet s'appuie sur une approche hybride combinant le cycle en V pour la gouvernance globale et la méthode Agile/Scrum pour l'itération technique. Cette approche permet de concilier rigueur de spécification et capacité d'adaptation face à la complexité des systèmes legacy et des modèles d'IA.

Le développement a été découpé en sprints de deux à quatre semaines, avec les rituels suivants :

- **Backlog produit** : centralisation des fonctionnalités (agents, API, ingestion, tests).
- **Sprints thématiques** : intake, collecteurs, raisonnement, guardrail, reporting, notification.
- **Daily stand-ups** : synchronisation rapide avec les encadrants et l'équipe technique.
- **Démos et rétrospectives** : validation des livrables et amélioration continue.

1.3.2 Planification

Le tableau ?? résume les grandes phases du projet.

Phase	Description	Durée
Analyse	Étude du SI CCU, APIs TMF et sources de données	2 semaines
Conception	Architecture agentique, modèle de données Neo4j, flux	3 semaines
Réalisation	Implémentation des agents, API, ingestion	5 semaines
Validation	Tests pytest, golden incidents, recette	2 semaines
Documentation	Rédaction du rapport et démonstration	2 semaines

TABLE 1.3 : Planification macro du projet

Conclusion

Ce premier chapitre a présenté le contexte général du projet, l'organisme d'accueil, la problématique du diagnostic technique CCU, les objectifs et la méthodologie adoptée. Le chapitre suivant détaille la conception de l'architecture agentique et du graphe de connaissances.

Conception de l'Architecture Agentique

Introduction

Ce chapitre présente la conception de l'architecture agentique du diagnostic technique CCU. Il décrit le pipeline global orchestré par LangGraph, le rôle de chaque agent, le modèle de données partagé, le graphe Neo4j et le mécanisme de recherche augmentée par graphe (GraphRAG).

2.1 Vue d'ensemble du pipeline

L'architecture repose sur un graphe d'état (state graph) implémenté avec **LangGraph**. Chaque nœud du graphe correspond à un agent spécialisé qui lit et écrit dans un état partagé. Le flux de traitement d'un incident suit le cheminement suivant :

```
START → intake → collectors → root_cause → ticket_manager  
→ remediation_explainer → content_guardrail → report_generator →  
notifier → END
```

La figure ?? illustre ce pipeline agentique.

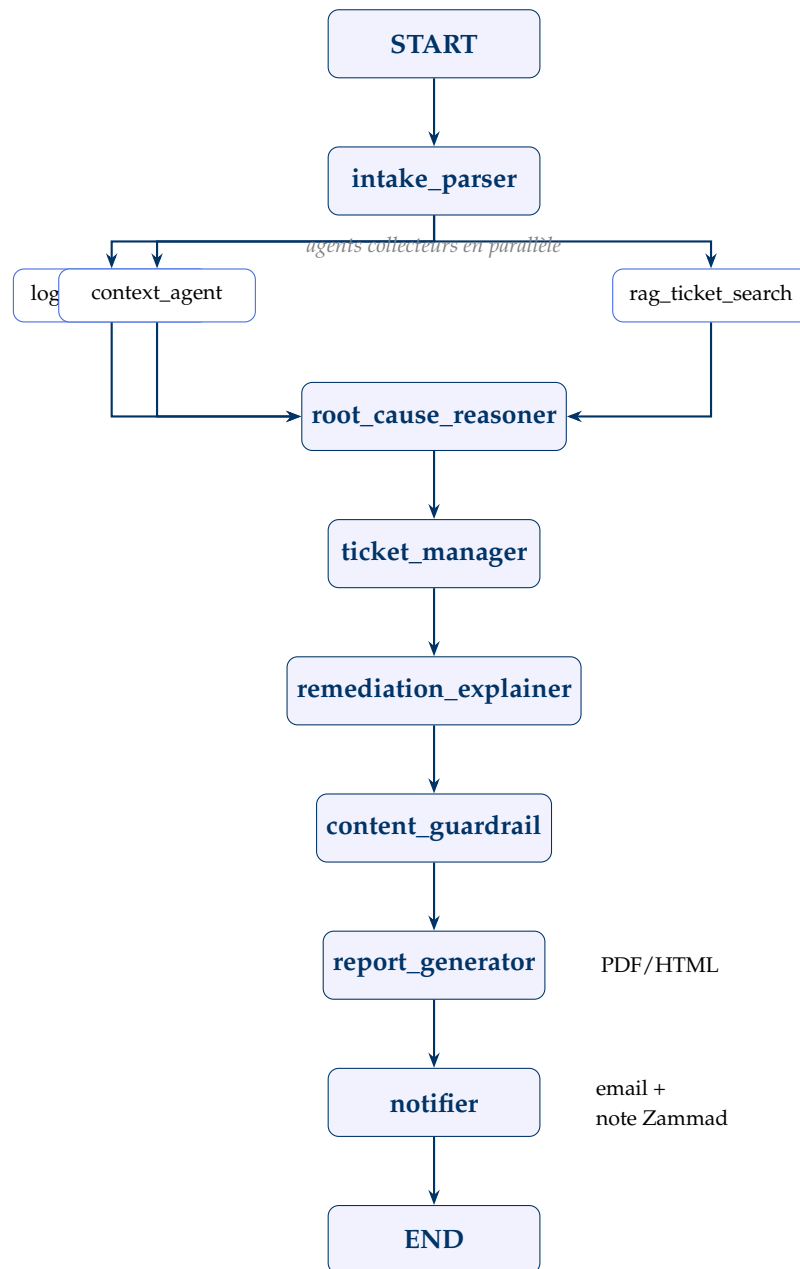


FIGURE 2.1 : Pipeline agentique du diagnostic technique CCU

2.2 Modèle de données partagé

L'état du graphe est matérialisé par la classe `GraphState` définie dans `shared/state.py`. Elle utilise **Pydantic** pour valider les données à chaque transition. Les champs principaux sont :

- `incident` : description brute de l'incident.
- `parsed_incident` : structure normalisée (client, service, commande, type, priorité).
- `logs`, `customer_context`, `similar_tickets` : résultats des agents collecteurs.
- `root_cause` : cause racine et niveau de confiance.
- `remediation_explanation` : explication de la remédiation proposée.
- `ticket_mapping` : mapping ou création du ticket.
- `sanitized_*` : champs filtrés par le guardrail.

- `report_path`, `email_sent`, `zammad_note_added` : sorties finales.
- `traces` : journal d'audit de l'exécution.

La classe `ParsedIncident` formalise les attributs extraits lors de l'intake :

Attribut	Description
<code>service_id</code>	Identifiant du service concerné (ex. <code>svc-fiber-12345</code>)
<code>order_id</code>	Identifiant de la commande (ex. <code>ord-2026-001</code>)
<code>customer_id</code>	Identifiant du client (ex. <code>acc-12345</code>)
<code>incident_type</code>	Type d'incident : fibre, sim, mobile, lan, billing, autre
<code>description</code>	Texte décrivant l'incident
<code>priority</code>	Priorité P1, P2 ou P3

TABLE 2.1 : Attributs de l'incident normalisé

2.3 Les agents du pipeline

2.3.1 Intake parser

L'agent `intake_parser` a pour mission de normaliser l'incident brut. Il combine une phase de parsing heuristique par expressions régulières et une validation/ajustement par LLM structuré lorsque le modèle est disponible. En mode `MOCK_LLM=true`, le système fonctionne de manière déterministe sans dépendance externe.

2.3.2 Collecteurs parallèles

Les trois agents collecteurs s'exécutent en parallèle dans le nœud `collectors` :

- `logs_investigator` : synthétise les logs réseau et applicatifs pertinents.
- `context_agent` : récupère le contexte client et contractuel dans le CRM.
- `rag_ticket_search` : recherche des tickets historiques similaires dans Neo4j.

Le parallélisme est mis en œuvre via `asyncio.gather`, avec un fallback `ThreadPoolExecutor` en cas de boucle déjà active.

2.3.3 Root cause reasoner

L'agent `root_cause_reasoner` corrèle les sources collectées pour établir une cause racine. Il retourne un niveau de confiance (faible, moyenne, forte) et une liste de `source_ids`. Si aucune source suffisante n'est disponible, il retourne une cause indéterminée avec une confiance faible, ce qui constitue un mécanisme de refus d'hallucination.

2.3.4 Ticket manager

L'agent `ticket_manager` détermine s'il faut créer un nouveau ticket ou rattacher l'incident à un ticket existant. Il s'appuie sur le client, le service et les tickets similaires déjà présents.

2.3.5 Remediation explainer

L'agent `remediation_explainer` produit un texte informatif décrivant les actions recommandées. Conformément au principe de sécurité du projet, ces actions ne sont jamais exécutées automatiquement.

2.3.6 Content guardrail

Le guardrail `pii_sanitizer` filtre les informations personnelles identifiables (PII) avant toute génération de rapport ou notification externe. Il classe également les actions selon une liste blanche pour refuser tout comportement non autorisé.

2.3.7 Report generator et notifier

L'agent `report_generator` produit un rapport PDF/HTML à partir d'un template Jinja2, avec WeasyPrint ou un fallback HTML. L'agent `notifier` envoie l'email et ajoute une note interne dans Zammad.

2.4 Graphe de connaissances Neo4j

2.4.1 Modèle de données graphe

Le graphe Neo4j stocke les entités métiers et leurs relations. Il sert à la fois de base de connaissances structurée et de support à la recherche vectorielle. Les principaux nœuds sont : `Client`, `Subscription`, `Order`, `Product`, `Ticket` et `LogEvent`.

La figure ?? représente le modèle de données graphe.

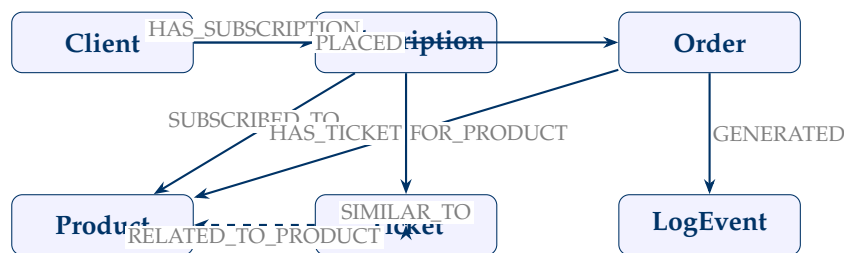


FIGURE 2.2 : Modèle de données du graphe Neo4j

2.4.2 Contraintes et index vectoriel

Le fichier `graph/schema.cypher` définit les contraintes d'unicité sur chaque entité et crée un index vectoriel sur les embeddings des tickets. Cet index permet d'effectuer une recherche de similarité cosine nativement dans Neo4j.

Listing 2.1 : Extrait du `schema.cypher`

```

1 CREATE CONSTRAINT ticket_id IF NOT EXISTS
2 FOR (t:Ticket) REQUIRE t.id IS UNIQUE;
3
4 CREATE VECTOR INDEX ticket_embeddings IF NOT EXISTS
5 FOR (t:Ticket) ON (t.embedding)
6 OPTIONS {indexConfig: {

```

```
7   `vector.dimensions` : __VECTOR_DIM__,  
8   `vector.similarity_function` : 'cosine'  
9   };
```

2.4.3 Recherche GraphRAG

La requête principale `search_similar_incidents`, définie dans `graph/queries.py`, combine deux mécanismes :

1. **Recherche vectorielle** : l'incident courant est vectorisé par un provider d'embedding, puis Neo4j retourne les tickets les plus proches en termes de cosine similarity.
2. **Filtre de proximité de graphe** : seuls les tickets liés au même produit ou au même type de commande que l'incident courant sont conservés.

Cette approche **GraphRAG** améliore la précision par rapport à une simple recherche sémantique en intégrant la structure relationnelle du graphe.

2.4.4 Provider d'embeddings

L'interface `EmbeddingProvider` définie dans `graph/embedding_provider.py` permet de changer de provider d'embedding (Ollama, sentence-transformers, OpenAI, Voyage AI) sans modifier le reste du code. Le provider Ollama est utilisé par défaut avec le modèle `mxbai-embed-large:latest`.

2.5 API REST et interface utilisateur

2.5.1 FastAPI et streaming SSE

L'API est développée avec **FastAPI**. La route `/diagnose/stream` retourne un flux d'événements Server-Sent Events (SSE), chaque événement correspondant à un nœud traversé par le graphe LangGraph. Cette approche permet au client (chatbot ou outil externe) de suivre la progression du diagnostic en temps réel.

2.5.2 Chatbot Next.js

L'interface utilisateur est un chatbot développé avec **Next.js**. Il affiche la trace d'exécution des agents, le badge de création du ticket et la carte de rapport avec le lien de téléchargement PDF.

Conclusion

Ce chapitre a présenté l'architecture agentique du projet : un pipeline LangGraph modulaire, un état partagé validé par Pydantic, des agents spécialisés et un graphe de connaissances Neo4j exploitant la recherche GraphRAG. Le chapitre suivant détaille l'implémentation technique de ces composants.

Réalisation et Implémentation

Introduction

Ce chapitre détaille la réalisation technique du projet. Il présente la stack utilisée, l’implémentation du pipeline LangGraph, des extraits de code représentatifs des agents, la configuration et les mécanismes de sécurité mis en place.

3.1 Stack technique

Le projet est développé en Python 3.11 et s’appuie sur un ensemble de bibliothèques et outils open source :

Composant	Rôle
LangGraph	Orchestration agentique sous forme de graphe d’état
FastAPI	API REST et streaming Server-Sent Events
Neo4j	Base de données graphe et index vectoriel
Ollama	Exécution locale des LLM et modèles d’embedding
Pydantic	Validation des schémas structurés
pytest	Tests unitaires et d’intégration
Docker Compose	Conteneurisation de l’API et des services
Jinja2 / WeasyPrint	Génération de rapports PDF/HTML
Next.js	Interface utilisateur chatbot

TABLE 3.1 : Stack technique du projet

3.2 Implémentation du pipeline

3.2.1 Construction du graphe

La classe `DiagnosticPipeline` dans `orchestrator/pipeline.py` construit le graphe LangGraph en déclarant les nœuds et les arêtes. Le nœud `collectors` encapsule le parallélisme des trois agents collecteurs.

Listing 3.1 : Construction du graphe LangGraph

```

1 from langgraph.graph import StateGraph, START, END
2
3 class DiagnosticPipeline:
4     def __init__(self) -> None:
5         self.graph = self._build_graph()
6
7     def _build_graph(self) -> StateGraph:
8         builder = StateGraph(GraphState)
9         builder.add_node("intake", run_intake)
10        builder.add_node("collectors", _collect)
11        builder.add_node("root_cause", run_root_cause)
12        builder.add_node("ticket_manager", run_ticket_manager)
13        builder.add_node("remediation_explainer", run_remediation_explanation)
14        builder.add_node("content_guardrail", run_content_guardrail)
15        builder.add_node("report_generator", run_report_generator)
16        builder.add_node("notifier", run_notifier)
17
18        builder.add_edge(START, "intake")
19        builder.add_edge("intake", "collectors")
20        builder.add_edge("collectors", "root_cause")
21        builder.add_edge("root_cause", "ticket_manager")
22        builder.add_edge("ticket_manager", "remediation_explainer")
23        builder.add_edge("remediation_explainer", "content_guardrail")
24        builder.add_edge("content_guardrail", "report_generator")
25        builder.add_edge("report_generator", "notifier")
26        builder.add_edge("notifier", END)
27        return builder.compile()

```

3.2.2 Parallélisme des collecteurs

La fonction `_collect` tente d'abord une exécution asynchrone via `asyncio.gather`. Si une boucle d'événements est déjà active, elle bascule vers un `ThreadPoolExecutor`.

Listing 3.2 : Parallélisme des collecteurs

```

1 async def _collect_async(state: GraphState) -> dict[str, Any]:
2     results = await asyncio.gather(
3         asyncio.to_thread(run_logs, state),
4         asyncio.to_thread(run_context, state),
5         asyncio.to_thread(run_rag, state),
6     )
7     merged: dict[str, Any] = {}
8     for r in results:
9         merged.update(r)
10    return merged

```


3.3 Exemples d'agents

3.3.1 Intake parser

L'agent d'intake utilise des expressions régulières pour extraire les identifiants métier, puis détermine le type d'incident et la priorité par des règles heuristiques.

Listing 3.3 : Parsing heuristique de l'incident

```

1 class IntakeParserAgent:
2     def _heuristic_parse(self, incident: dict[str, Any]) -> IncidentSchema:
3         text = incident.get("description", "") + " " + incident.get("title", "")
4         service_id = self._extract_field(text, [
5             r"service[_-]?id\s*[:=]\s*([a-z0-9-]+)",
6             r"\b(svc-[a-z0-9-])\b",
7         ])
8         order_id = self._extract_field(text, [
9             r"\b(ord-[0-9-])\b",
10        ])
11        customer_id = self._extract_field(text, [
12            r"\b(acc-[0-9-])\b",
13        ])
14        # Type et priorité déterminées par mots-clés
15        ...

```

3.3.2 Root cause reasoner

L'agent de raisonnement implémente une logique déterministe qui s'appuie sur les sources collectées. Si un ticket similaire fournit une cause racine, celle-ci est retenue. Sinon, l'agent applique des règles par type d'incident. En l'absence de source suffisante, il retourne une cause indéterminée.

Listing 3.4 : Refus d'hallucination dans le root cause reasoner

```

1 def _deterministic_cause(self, state: GraphState) -> RootCauseSchema:
2     sources, _ = self._collect_sources(state)
3     if not sources:
4         return RootCauseSchema(
5             confidence="faible",
6             cause="indeterminée",
7             explanation="Aucune source suffisante ...",
8             source_ids=[],
9         )
10    # Corrélation avec le ticket similaire le plus proche
11    rag = state.similar_tickets or {}
12    similar = rag.get("similar_tickets", [])
13    if similar:
14        best = min(similar, key=lambda t: t.get("distance", 1.0))
15        cause = best.get("root_cause", "indeterminée")
16    ...

```

3.3.3 Requête GraphRAG

La requête Cypher suivante illustre la combinaison de la recherche vectorielle native Neo4j avec un filtre de proximité de graphe.

Listing 3.5 : Recherche GraphRAG dans Neo4j

```

1 CALL db.index.vector.queryNodes($index_name, $candidate_limit, $query_embedding)

```

```

2 YIELD node AS t, score
3 WHERE score >= $threshold
4   AND (size(product_names) = 0 OR EXISTS {
5     MATCH (t) -[:RELATED_TO_PRODUCT]->(p:Product)
6     WHERE p.name IN product_names
7   })
8 RETURN t.id AS ticket_id, t.summary AS summary,
9        t.root_cause AS root_cause, t.resolution AS resolution, score
10 ORDER BY score DESC
11 LIMIT $top_k

```

3.4 Configuration et variables d'environnement

3.4.1 Fichier .env

Le projet utilise un fichier `.env` pour la configuration. Le mode `MOCK_LLM=true` permet de démarrer sans Ollama, en utilisant une logique déterministe.

Variable	Description
MOCK_LLM	Mode LLM simulé (true/false)
OLLAMA_BASE_URL	URL du service Ollama local
OLLAMA_MODEL	Modèle de langage local
NEO4J_URI	URI de connexion Neo4j
NEO4J_USER / NEO4J_PASSWORD	Credentials Neo4j
EMBEDDING_PROVIDER	Provider d'embedding (ollama, ...)
VECTOR_SIMILARITY_THRESHOLD	Seuil de similarité cosine

TABLE 3.2 : Principales variables d'environnement

3.4.2 Makefile

Le `Makefile` fournit les commandes essentielles du projet :

- `make install` : installation des dépendances Python.
- `make seed` : seed de Neo4j (schéma, mocks, embeddings).
- `make test` : exécution des tests pytest.
- `make api` : lancement de l'API FastAPI.
- `make docker-up` / `make docker-down` : gestion des conteneurs.

3.5 Sécurité et guardrails

3.5.1 Principe de non-exécution automatique

Le projet respecte une règle fondamentale : **aucune action technique n'est jamais exécutée automatiquement** sur un système réel. L'agent produit des diagnostics, des explications et des rapports,

mais toute action corrective reste sous le contrôle d'un opérateur humain.

3.5.2 Guardrail de contenu

Le guardrail de contenu filtre les informations personnelles identifiables (PII) avant toute génération de rapport ou notification. Il s'appuie sur une logique de whitelist pour les actions autorisées et refuse toute action hors périmètre.

3.5.3 Refus d'hallucination

Le `root_cause_reasoner` retourne systématiquement une cause indéterminée avec une confiance faible lorsqu'il ne peut citer aucune source. Cette mesure limite les diagnostics inventés par le modèle.

Conclusion

Ce chapitre a détaillé la réalisation technique du projet : stack Python, orchestration LangGraph, implémentation des agents, requêtes GraphRAG, configuration et guardrails de sécurité. Le chapitre suivant traite de la validation, du déploiement et de la démonstration de bout en bout.

Validation, Déploiement et Démonstration

Introduction

Ce chapitre présente la stratégie de validation du projet, les tests automatisés mis en place, le déploiement via Docker Compose, ainsi qu'un scénario de démonstration de bout en bout.

4.1 Stratégie de test

4.1.1 Tests unitaires et d'intégration

Le projet utilise **pytest** pour valider le fonctionnement des agents, de l'ingestion Neo4j et des requêtes GraphRAG. La configuration de pytest est définie dans `pyproject.toml` avec le mode asynchrone activé.

Listing 4.1 : Configuration pytest dans pyproject.toml

```
1 [tool.pytest.ini_options]
2 testpaths = ["tests"]
3 pythonpath = ["."]
4 asyncio_mode = "auto"
```

Les tests couvrent les domaines suivants :

- **Intake** : validation du parsing heuristique.
- **Contexte** : récupération du contexte client.
- **Guardrail** : vérification du filtrage PII.
- **Pipeline** : exécution complète du graphe en mode mock.
- **Graph ingestion** : compteurs de nœuds et relations après seed.
- **GraphRAG** : recherche de tickets similaires.

4.1.2 Tests Neo4j

Les tests `test_graph_ingestion.py` et `test_rag.py` nécessitent une instance Neo4j accessible. Ils seedent automatiquement le graphe et vérifient :

- La création correcte des nœuds `Client`, `Subscription`, `Order`, `Product`, `Ticket` et `LogEvent`.
- L'existence des relations attendues.
- Le fonctionnement de la recherche vectorielle avec le seuil de similarité configuré.

4.1.3 Golden incidents

Le répertoire `evaluation/golden_incidents/` contient un ensemble de cas de test attendus. Le runner `eval_runner.py` rejoue ces incidents, compare les résultats aux attentes et produit un rapport pass/fail avec une accuracy globale. Le processus retourne un code de sortie 0 si tous les cas passent.

Cas	Objectif	Résultat attendu
Coupure fibre	Valider la corrélation logs + ticket similaire	Cause fibre identifiée
Timeout CRM	Valider le contexte client et logs applicatifs	Cause CRM identifiée
SIM non provisionnée	Valider le type d'incident mobile	Cause HLR identifiée
VLAN mismatch	Valider le type d'incident LAN	Cause VLAN identifiée
Billing	Valider le fallback faible confiance	Cause indéterminée

TABLE 4.1 : Exemples de cas golden incidents

4.2 Déploiement

4.2.1 Docker Compose

Le fichier `docker/docker-compose.yml` permet de démarrer l'ensemble des services : API FastAPI, Neo4j, Zammad, Mailhog, Splunk et Prism. Cette infrastructure facilite la démonstration de bout en bout sans dépendre d'un environnement externe.

Listing 4.2 : Démarrage de l'infrastructure

```
1 cd docker
2 docker compose up --build
```

L'API est alors accessible sur `http://127.0.0.1:8000`, Neo4j sur `http://localhost:7474`, Zammad sur `http://localhost:3000` et Mailhog sur `http://localhost:8025`.

4.2.2 Seed des données

Une fois l'infrastructure démarrée, les commandes suivantes permettent de peupler la base Neo4j et Zammad :

Listing 4.3 : Seed des données

```
1 make seed
2 make seed-tickets
3 make seed-zammad
```

4.3 Scénario de démonstration

4.3.1 Description du cas

Un opérateur reçoit l'incident suivant, exprimé en langage naturel :

Client acc-12345, service svc-fiber-12345, commande ord-2026-001. Coupure Internet fibre.

4.3.2 Déroulement du diagnostic

Le pipeline exécute successivement les étapes suivantes :

1. **Intake** : extraction de `customer_id=acc-12345, service_id=svc-fiber-12345, order_id=ord-2026-001, type fibre, priorité P2`.
2. **Collecte** : logs OLT, contexte client CRM et tickets similaires de coupure fibre en parallèle.
3. **Root cause** : cause racine Perte de signal fibre optique avec confiance moyenne ou forte.
4. **Ticket manager** : création du ticket `TICK-CCU-XXXX`.
5. **Remediation** : proposition d'actions informatives (vérification de la paire, test optique).
6. **Guardrail** : vérification PII.
7. **Report generator** : génération du PDF `INC-CCU-XXXX.pdf`.
8. **Notifier** : envoi d'un email et ajout d'une note interne dans Zammad.

4.3.3 Observation des résultats

La démonstration peut être observée à plusieurs endroits :

- **Chatbot Next.js** : affichage de la trace agent et du lien de téléchargement du rapport.
- **Mailhog** : réception de l'email avec le PDF en pièce jointe.
- **Zammad** : ticket créé avec une note interne indiquant l'envoi du rapport.
- **Neo4j Browser** : visualisation des nœuds et relations du graphe avec le style GRASS fourni.

Conclusion

Ce chapitre a présenté la stratégie de validation du projet, fondée sur des tests automatisés et des cas golden incidents, ainsi que le déploiement via Docker Compose et un scénario de démonstration de bout en bout. Le projet démontre la faisabilité d'un agent IA de diagnostic technique pour le SI CCU, tout en respectant les contraintes de sécurité et de traçabilité.

Conclusion Générale

Au terme de ce projet de fin d'études, réalisé au sein d'Inetum Offshore Maroc, les objectifs de conception et de réalisation d'un agent IA de diagnostic technique pour le système d'information CCU ont été atteints. Le travail mené démontre qu'il est possible de combiner orchestration agentique, graphe de connaissances et modèles de langage pour accélérer le diagnostic d'incidents télécom tout en maîtrisant les risques.

La première contribution de ce projet est architecturale. Le pipeline conçu autour de **LangGraph** articule de manière modulaire les étapes d'analyse d'un incident : parsing, collecte parallèle des sources, raisonnement sur la cause racine, gestion du ticket, génération de remédiation, guardrail de contenu, production de rapport et notification. Cette décomposition en agents spécialisés facilite l'évolution, le test et l'audit du système.

La seconde contribution est technique. L'intégration de **Neo4j** comme base de graphe et d'index vectoriel permet de stocker les entités métier et d'effectuer une recherche de tickets similaires combinant similarité sémantique et proximité de graphe. L'utilisation de **FastAPI**, de **Pydantic** et d'une interface d'embedding extensible assure un socle robuste et industrialisable.

Enfin, la démarche de validation, fondée sur des tests automatisés avec **pytest** et un jeu de **golden incidents**, garantit la reproductibilité et la mesure de la performance du système. Le déploiement via **Docker Compose** et le scénario de bout en bout valident la faisabilité opérationnelle de la solution.

Sur le plan personnel, ce projet m'a permis d'approfondir les concepts d'IA générative appliquée au support, de conception multi-agents, de bases de données graphe et de déploiement d'applications LLM en contexte entreprise. Les perspectives d'évolution incluent l'enrichissement du graphe avec des données temps réel, l'amélioration du raisonnement par LLM local et l'intégration de mécanismes de feedback humain pour affiner continuellement la qualité du diagnostic.

Bibliographie

- [1] P. Goyal, V. Chettiar, et al. Graphrag : Unlocking llm discovery on narrative private data. *Microsoft Research Blog*, 2023.
- [2] LangChain Inc. Langgraph : Build stateful, multi-agent applications with llms. <https://langchain-ai.github.io/langgraph/>, 2024. Consulté en 2026.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [4] Neo4j Inc. Neo4j graph database platform. <https://neo4j.com/>, 2024. Consulté en 2026.
- [5] Ollama Inc. Ollama : Get up and running with large language models locally. <https://ollama.com/>, 2024. Consulté en 2026.
- [6] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022.
- [7] Pydantic Services Inc. Pydantic : Data validation using python type hints. <https://docs.pydantic.dev/>, 2024. Consulté en 2026.
- [8] Sebastián Ramírez. Fastapi : Modern, fast web framework for building apis with python. <https://fastapi.tiangolo.com/>, 2024. Consulté en 2026.
- [9] TM Forum. Tm forum apis : Tmf620 product catalog and tmf622 product ordering. <https://www.tmforum.org/oda/open-apis/>, 2024. Consulté en 2026.